

Heaps: Complexities

1. Recall

If there are already n elements in a heap:

1. One insertion takes $O(\log_2 n)$ time.
2. One deletion takes $O(\log_2 n)$ time.

Reason: a heap is a complete binary tree of height $O(\log_2 n)$.

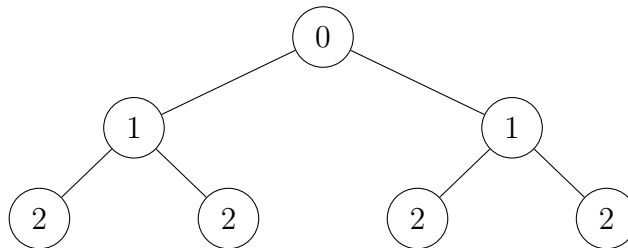
2. Repeated Insertions

Suppose we build a heap by inserting elements one by one.

Each new element:

1. Is placed at the next free position.
2. Then may move upward using **bubble up**.

If inserted at depth i , then in worst case it may perform i swaps.



(Node labels represent depth.)

Depth	Max Swaps	Number of Nodes
0	0	1
1	1	2
2	2	4
3	3	8
$\vdots$	$\vdots$	$\vdots$
$\log_2(\frac{n}{8})$	$\log_2(\frac{n}{8})$	$\frac{n}{8}$
$\log_2(\frac{n}{4})$	$\log_2(\frac{n}{4})$	$\frac{n}{4}$
$\log_2(\frac{n}{2})$	$\log_2(\frac{n}{2})$	$\frac{n}{2}$

At depth i :

- Number of nodes $= 2^i$
- Maximum swaps per node $= i$
- Total work at that depth $= i \cdot 2^i$

Total cost is:

$$\left(\frac{n}{2}\right) (\log_2(\frac{n}{2})) + \left(\frac{n}{4}\right) (\log_2(\frac{n}{4})) + \left(\frac{n}{8}\right) (\log_2(\frac{n}{8})) + \dots + 2 \cdot 1 + 1 \cdot 0$$

Factor out n :

$$n \left[\frac{1}{2} (\log_2(\frac{n}{2})) + \frac{1}{4} (\log_2(\frac{n}{4})) + \frac{1}{8} (\log_2(\frac{n}{8})) + \dots \right]$$

Separate terms:

$$n \left[\log_2 n \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right) - \left(\frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \dots \right) \right]$$

2.1 First series

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 1$$

2.2 Second series

Let

$$S = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \dots$$

Multiply by 2:

$$2S = 1 + \frac{1}{2} \cdot 2 + \frac{1}{4} \cdot 3 + \frac{1}{8} \cdot 4 + \dots$$

Subtract S :

$$2S - S = 1 + \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

Using geometric series:

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 1$$

So:

$$S = 2$$

Substituting back:

$$T(n) = n [\log_2 n \cdot 1 - 2]$$

$$T(n) = n \log_2 n - 2n$$

In Big-O notation:

$$O(n \log_2 n)$$

$$\text{Building heap via repeated insertion} = O(n \log_2 n)$$

3. Repeated Deletions (Heap Sort)

The same depth-based perspective used for insertions also applies to deletions, because both operations are constrained by the height of the heap, which is $O(\log_2 n)$. The only difference is the direction of movement: insertion moves upward using bubble-up, while deletion moves downward using bubble-down. However, in both cases, the total possible movement of any element is bounded by its depth in the heap.

During deletion, the root is removed and replaced with the last element, after which the heap property is restored by moving this element downward. If an element finally settles at depth i , then in the worst case it performs i swaps. Since a complete binary heap has 2^i nodes at depth i , the cost structure across levels remains identical to insertion, with only the interpretation reversed.

Depth in Heap	Max Swaps (Bubble Down)	Number of Nodes
$\log_2 n - 1$	$\log_2 n - 1$	$\frac{n}{2}$
$\log_2 n - 2$	$\log_2 n - 2$	$\frac{n}{4}$
$\log_2 n - 3$	$\log_2 n - 3$	$\frac{n}{8}$
$\vdots$	$\vdots$	$\vdots$
2	2	4
1	1	2
0	0	1

Thus, even in deletion, each level contributes work proportional to $i \cdot 2^i$, and the same structural distribution of cost appears as in insertion. Since the height of the heap is $O(\log_2 n)$, each deletion takes $O(\log_2 n)$ time, and performing n deletions (as in heap sort) results in a total complexity of $O(n \log_2 n)$.